

Reviewer Pack — Minimal Rank of Tropicalized Classical Groups Bounds ACC^0 Ci...

Entry b3afa1bf7242 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 20:25:11 UTC

Minimal Rank of Tropicalized Classical Groups Bounds ACC^0 Circuit Width

Entry ID: b3afa1bf7242 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 20:25:11 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (Classical Groups) **Field B** (complexity object): Complexity Theory: ACC^0 Circuit Complexity

Statement:

For every classical group G , the minimal rank of its tropicalization in the max-plus semiring is a lower bound on the width of an ACC^0 circuit computing an isomorphism class of G . Equivalently, there exists a constant c such that for all G and any ACC^0 circuit C with width $w(C)$, if the minimal rank of G 's tropicalization is at least c , then $w(C) \geq c * |G|$.

Rationale (proposer's reasoning):

The classical groups are fundamental algebraic objects with rich structure. Their tropicalizations provide a combinatorial view that may reveal non-trivial connections to circuit complexity, particularly in ACC^0 classes where circuits have bounded fan-in and -out. A lower bound on the rank of the tropicalization could translate into a lower bound on the circuit width, potentially providing new insights into ACC^0 complexity.

Taxonomy category: TROPICAL_GEOMETRY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d07df7718a8bf6cb

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Spearman rank correlation coefficient between the minimal ranks of tropicalizations and the widths of ACC^0 circuits exceeds 0.5 for at least 30 out of 40 seeds, with no seed producing a negative correlation.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    for i in range(rows):
        max_row = i + max(range(i, rows), key=lambda x: abs(matrix[x][i]))
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        factor = Fraction(1, matrix[i][i])
        for j in range(cols):
            matrix[i][j] *= factor
        for k in range(rows):
            if k != i:
                factor = -matrix[k][i]
                for j in range(cols):
                    matrix[k][j] += factor * matrix[i][j]
    return matrix

def determinant(matrix):
    rows, cols = len(matrix), len(matrix[0])
    if rows != cols:
        raise ValueError("Matrix must be square")
    det = Fraction(1)
    for i in range(rows):
        max_row = i + max(range(i, rows), key=lambda x: abs(matrix[x][i]))
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        factor = -matrix[i][i]
        for k in range(rows):
            if k != i:
                factor *= matrix[k][i]
        det *= factor
    return det

def run_trial(seed: int) -> dict:
    random.seed(seed)
```

```

# Generate a random classical group (for simplicity, use a 2x2 matrix)
n = 40
group = [[random.randint(1, 5) for _ in range(n)] for _ in range(n)]

# Tropicalize the group
tropicalized_group = [[max(a, b) for b in row] for row in group]

# Calculate the rank of the tropicalized group using Gaussian elimination
rank = sum(1 for row in gaussian_elimination(tropicalized_group) if any(row))

# Construct an ACC0 circuit (for simplicity, use a random width)
acc0_circuit_width = random.randint(1, 10)

# Measure the metric: rank of tropicalization vs. width of ACC0 circuit
metric_name = "Rank vs. Circuit Width"
metric_value = rank / acc0_circuit_width

# Check if the conjecture holds for this seed
conjecture_holds = metric_value >= 1.5 # Placeholder bound, adjust as needed
counterexample = "" if conjecture_holds else f"Rank {rank} < Width {acc0_circuit_width}"

return {
    "metric_name": metric_name,
    "metric_value": metric_value,
    "instances_tested": n * n,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    # Compute mean and std of metric_value
    if not results:
        print("RESULT: INCONCLUSIVE no_results")
    else:
        total_metric = sum(result["metric_value"] for result in results)
        count = len(results)
        mean = total_metric / count
        variance = sum((result["metric_value"] - mean) ** 2 for result in results) / count
        std = math.sqrt(variance)

    # Check fraction of seeds where conjecture holds
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / count

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(result["seed"] for result in results if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Rank < Width\" first_failing_seed={first_failing_seed}")

```

```

else:
    print("RESULT: INCONCLUSIVE insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9bd0365d.py", line 85, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9bd0365d.py", line 56, in run_trial
    tropicalized_group = [[max(a, b) for b in row] for row in group]
                           ^
NameError: name 'a' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to provide a result for the conjecture. | next: Investigate and fix the crash in the test code to ensure it can run to completion and provide results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13332
2	preregistration	ollama_remote	glm4:latest	0	0	5609
3	novelty	ollama_remote	glm4:latest	0	0	4798
4	novelty	ollama_remote	glm4:latest	0	0	5176
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13409
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9227
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9489
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11400
9	judge	ollama_remote	glm4:latest	0	0	10668

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 83109 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/b3afa1bf7242.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b3afa1bf7242.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b3afa1bf7242.tar.gz` (if generated)